

# Cálculo paralelo estocástico de PI usando atomic e reduction

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

long N = 10000000000;

// ESSA É PARTE DA ATIVIDADE 8, IMPLEMENTAÇÃO COM MELHOR RESULTADO
void pi_v3() {
    long global_hits = 0;

    double start, end, elapsed;

    start = omp_get_wtime();

#pragma omp parallel
    {
        long local_hits = 0;
        unsigned int seed = (unsigned int)omp_get_thread_num();
#pragma omp for
        for (long i = 0; i < N; i++) {
            double x = (double)rand_r(&seed) / RAND_MAX;
            double y = (double)rand_r(&seed) / RAND_MAX;
            if (x * x + y * y <= 1.0) {
                local_hits++;
            }
        }
#pragma omp critical
        global_hits += local_hits;
    }

    end = omp_get_wtime();

    elapsed = end - start;

    double pi = 4.0 * (double)global_hits / (double)N;

    printf("PI_V3 >>> Elapsed time: %f || Estimativa: %f\n", elapsed, pi);
}

// PARTE DA TAREFA 10
void pi_v5() {
    long global_hits = 0;

    double start, end, elapsed;

    start = omp_get_wtime();
#pragma omp parallel
```

```

{
    long local_hits = 0;
    unsigned int seed = omp_get_thread_num();
#pragma omp for
    for (long i = 0; i < N; i++) {
        double x = (double)rand_r(&seed) / RAND_MAX;
        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0)
            local_hits++;
    }
#pragma omp atomic
    global_hits += local_hits;
}

end = omp_get_wtime();

elapsed = end - start;

double pi = 4.0 * (double)global_hits / (double)N;

printf("PI_V5 >>> Elapsed time: %f || Estimativa: %f\n", elapsed, pi);
}

void pi_v6() {
    long global_hits = 0;

    double start, end, elapsed;

    start = omp_get_wtime();
#pragma omp parallel
    {
        unsigned int seed = omp_get_thread_num();
#pragma omp for reduction(+ : global_hits)
        for (long i = 0; i < N; i++) {
            double x = (double)rand_r(&seed) / RAND_MAX;
            double y = (double)rand_r(&seed) / RAND_MAX;
            if (x * x + y * y <= 1.0)
                global_hits++;
        }
    }

    end = omp_get_wtime();

    elapsed = end - start;

    double pi = 4.0 * (double)global_hits / (double)N;

    printf("PI_V6 >>> Elapsed time: %f || Estimativa: %f\n", elapsed, pi);
}

int main() {
    pi_v3();
    pi_v5();
    pi_v6();
}

```

```
// Output
// → tasks clang task8.c -fopenmp -O3
// → tasks ./a.out
// PI_V3 >>> Elapsed time: 9.410929 || Estimativa: 3.141587
// PI_V5 >>> Elapsed time: 9.633228 || Estimativa: 3.141587
// PI_V6 >>> Elapsed time: 12.557908 || Estimativa: 3.141587
```